

NumPy Basics and Vectorized Thinking

This chapter is designed for tabular biomedical data such as heart rate, blood oxygen, and body temperature. The goal is not to memorize `for` loops, but to use NumPy operations, Boolean masks, `axis`-based reductions, shape alignment, missing-value handling, file I/O, cleaning pipelines, and standardized computational templates to express reusable API-driven logic.

Python 3 • NumPy

tutor style

figures included

FAQ included

9 core operation groups

00

Decision Table and Review Order

First decide which kind of task you have. The answer points directly to the NumPy tool you should use. Remembering this mapping is more useful than memorizing isolated APIs.

What you need to do	Recommended tool
Filter a one-dimensional array by a condition	Boolean mask: <code>mask = a > x</code> , <code>a[mask]</code>
Aggregate rows or columns in two-dimensional data	<code>np.mean/max/min(..., axis=)</code>
Turn a one-dimensional array into two dimensions	<code>reshape</code>
Align arrays of different lengths for computation	<code>broadcasting</code>
Modify only a copy and preserve the original array	<code>.copy()</code>
Find the position of the maximum or minimum	<code>argmax</code> / <code>argmin</code>
Sort values and return their positions	<code>argsort</code>
Inspect step-by-step changes	<code>diff</code>
Clamp values into a valid range	<code>clip</code>
Handle missing values	<code>np.nan + isnan + nanmean</code>
Run a reproducible random simulation	<code>np.random.default_rng(seed)</code>
Read and write arrays	<code>loadtxt/savetxt</code> , <code>load/save</code>
Combine multiple arrays	<code>concatenate/vstack/hstack</code>
Combine several metrics into one score	Matrix multiplication <code>@</code>

01 • START
Boolean masks
filter a group

02
axis reductions
2D data

03
reshape / view
shape and copies

04
argmax / sort
find positions

05
diff / clip
changes and ranges

06
NaN cleaning
missing values

07
Random simulation
reproducibility

08
Files + Z-score
anomaly detection

09
Combine + matrix
weighted scores

The essence of vectorization: replace "repeat this operation" with "apply this operation to the whole array." `mask = array > x` evaluates every element at once and returns a Boolean array of equal length, which can immediately index another array.

Boolean Masks: Filtering a 1D Array

Core concept

A **mask** is a True/False array with the same shape as the source array. Once created, it can index any other matching array, such as using a mask calculated from heart rate to retrieve patient IDs.

```
heart_rates = [72, 105, 88, 112, 96, 64]
```



```
mask = heart_rates > 100
```

```
[False, True, False, True, False, False]
```

```
patient_ids[mask]
```

```
[102, 104]
```

The mask and source arrays align element by element.

Code template

Template 1 • Filter with a Boolean mask

mask

```
def ids_over_threshold(ids, values, threshold):
    mask = values > threshold
    return ids[mask]

# ids: identifier array to return
# values: numeric decision array; same length as ids
# mask: Boolean array containing only True or False

normal_mask = (heart_rates <= 100) & (heart_rates >= 60)
follow_up_mask = (heart_rates > 100) | (temperatures >= 38.0)
```

Common mistake

Incorrect	Why it fails	Correct
<code>rates > 100 & oxygen < 95</code>	<code>&</code> has tighter precedence than comparisons, so Python evaluates <code>100 & oxygen</code> first.	<code>(rates > 100) & (oxygen < 95)</code>

Q. How is a mask different from a regular if statement?

`if` usually decides one value once. `mask = array > x` evaluates the entire array at once and returns a same-length True/False array that can directly index data. This is the core of vectorization.

2D Arrays and axis Reductions

Core concept

Rows = patients; columns = time points

Patient 1	72	74
Patient 2	98	102

axis=1 -> [73, 100] (average per patient)

axis=0 -> [85, 88] (average per day)

The result keeps the dimension not reduced.

axis=0 vs axis=1 + np.any

axis

```
heart_rate_matrix = np.array([
    [72, 74, 70],      # Patient 201, three days
    [98, 102, 104],    # Patient 202
])

np.mean(heart_rate_matrix, axis=0) # Daily means -> [85. 88. 87.]
np.mean(heart_rate_matrix, axis=1) # Patient means -> [72. 101.33]
np.any(heart_rate_matrix > 100, axis=1) # Did each patient ever exceed 100?
```

Memory rule: imagine the matrix as "patients by rows, time points by columns." **axis=0** moves down rows and leaves columns; **axis=1** moves across columns and leaves rows. The result keeps the dimension that was not reduced.

• Code template

Template 2 • Aggregate and filter

axis+mask

```
def high_average_ids(ids, matrix, threshold, axis=1):
    averages = np.mean(matrix, axis=axis)
    mask = averages > threshold
    return ids[mask]
```

Incorrect

Why it fails

Correct

Use **axis=0** for "average per person"

axis=0 reduces rows across patients and returns an average per day.

`np.mean(matrix, axis=1)`

Q. Should axis be memorized as 0 or 1?

Think of a patient-by-time matrix. **axis=0** travels downward and leaves columns; **axis=1** travels across and leaves rows. The dimension that survives is the one you keep.

03 reshape, Broadcasting, and View vs Copy

• Core concept

reshape + broadcasting + keepdims

shape

```
raw = np.array([72,74,70,73, 98,102,104,100, 88,90,86,89])
matrix = raw.reshape(3, 4) # 12 values -> 3 patients x 4 days
matrix + np.array([0, 1, -1, 0]) # A (4,) offset broadcasts to every row
row_means = np.mean(matrix, axis=1, keepdims=True) # shape (3,1)
centered = matrix - row_means # Each row subtracts its own mean
```



A view can modify the source array; a copy cannot

view/copy

```
first_row = matrix[0] # A view, not a new array
first_row[0] = 999 # Also modifies matrix
safe_row = matrix[0].copy() # Independent copy
safe_row[0] = 999 # matrix is unaffected
```

• Code template

Template 3 • Modify a copy safely

copy

```
def change_first_value_safely(matrix, replacement):
    copied_row = matrix[0].copy()
    copied_row[0] = replacement
    return matrix, copied_row # Original matrix unchanged
```

Incorrect

Why it fails

Correct

```
row = matrix[0]; row[0] = 999
```

Ordinary slicing creates a view that shares memory with the source.

```
row = matrix[0].copy()
```

Q. Why does a view affect the original array? Is this a bug?

No. It is a deliberate NumPy performance design. Slicing does not copy data by default; it presents another angle on the same memory, avoiding expensive copying. Use `.copy()` when independent data is required.

04 argmax, argmin, and argsort: Find Positions, Not Values

• Core concept

The return value is a position, not the value itself

position

```
averages = np.mean(heart_rate_matrix, axis=1)

np.argmax(averages)           # Position of the maximum value
ids[np.argmax(averages)]      # Retrieve the corresponding patient ID
np.argsort(averages)          # Positions ordered from smallest to largest
ids[np.argsort(averages)]     # Patient IDs in ascending order
```

• Code template

Template 4 • Find the ID associated with the maximum

argmax

```
def id_with_highest_average(ids, matrix):
    averages = np.mean(matrix, axis=1)
    return ids[np.argmax(averages)]
```

Incorrect

Why it fails

Correct

```
highest = np.argmax(averages) and print it
as the value
```

`argmax`, `argmin`, and `argsort` return positions or indices, not numeric values.

```
ids[np.argmax(averages)]
```

Use case: when the question asks not "what is the maximum?" but "who, which ID, or which row owns the maximum?" think of `argmax`, `argmin`, or `argsort`.

05 diff and clip: Changes and Range Constraints

• Core concept

Stepwise changes + clamping to a valid range

diff/clip

```
readings = np.array([70, 72, 78, 95, 110, 105, 90, 80])

np.diff(readings)
# [2, 6, 17, 15, -5, -15, -10]
# Length is one less; every value is "next minus previous."

np.clip(readings, 60, 100)
# Above 100 becomes 100; below 60 becomes 60.
```

Use case: use `diff` to inspect step-by-step changes and `clip` to constrain a sensor or instrument to a valid range. Both process the entire array in one step without a loop.

Missing Values: np.nan

Core concept

```
oxygen = [98.0, nan, 97.0, 91.0, nan, 96.0]
```



```
isnan -> [F, T, F, F, T, F] -> where(...)[0] -> [1, 4]
```

Locate missing values first; then decide how to handle them.

Detect + impute + combine conditions

nan

```
oxygen = np.array([98.0, np.nan, 97.0, 91.0, np.nan, 96.0])

np.isnan(oxygen)           # [False True False False True False]
np.where(np.isnan(oxygen))[0] # [1, 4], missing positions
mean_value = np.nanmean(oxygen) # Ignores nan
cleaned = oxygen.copy()
cleaned[np.isnan(cleaned)] = mean_value
~np.isnan(oxygen) # Negation means "not missing"
```

Code template

Template 5 • Impute missing values

fillna

```
def fill_missing_with_mean(values):
    cleaned = values.copy()
    mean_value = np.nanmean(cleaned)
    cleaned[np.isnan(cleaned)] = mean_value
    return cleaned
```

Incorrect

Why it fails

Correct

`np.mean(oxygen_with_nan)`

A single **nan** can contaminate the entire result.

`np.nanmean(oxygen_with_nan)`

Q. Are np.nan and Python None the same?

No. **np.nan** is a special floating-point value used in numeric arrays. **None** is Python's general-purpose "no value" object. Detect **np.nan** with `np.isnan(x)`; do not use `x is np.nan` or `x == np.nan`, because the latter is always False.

Random Numbers and Reproducibility

Core concept

Independent generator + reproducibility check

rng

```
rng = np.random.default_rng(seed=42) # Modern independent generator
rng.normal(loc=75, scale=8, size=6) # Normal distribution
rng.choice(ids, size=3, replace=False) # Sample without replacement

np.array_equal(rng1.integers(0, 100, 5),
               rng2.integers(0, 100, 5))
# Identical seeds must produce the same sequence.
```

Code template

Template 6 • Reproducible random simulation

reproducible

```
def simulate_values(count, seed):
    rng = np.random.default_rng(seed)
    return rng.normal(loc=75, scale=8, size=count)
```

```
np.random.seed(42)
np.random.normal(75, 8,
6)
```

Global random state can be changed by unrelated code in a large project, so "the same seed" may appear to produce a different result.

```
rng =
np.random.default_rng(42)
rng.normal(75, 8, 6)
```

Q. Why should a random number generator use a seed?

A seed fixes the generator's starting point. The same seed produces the same pseudo-random sequence. This lets another person rerun your code or GitHub project and reproduce exactly the same result.

08 File I/O and Z-score Anomaly Detection

• Core concept

Read + remove anomalies + save

file+zscores

```
readings = np.loadtxt(path, delimiter=",") # Text input usually returns float64
mean, std = readings.mean(), readings.std()
z_scores = (readings - mean) / std
anomaly_mask = np.abs(z_scores) > 2.0 # Large |z| means far from the group
clean = readings[~anomaly_mask]

np.savetxt(text_path, clean, delimiter=",") # Human-readable text
np.save(binary_path, clean) # Exact NumPy binary
loaded = np.load(binary_path)
np.array_equal(loaded, clean) # Round-trip verification
```

• Code template

Template 7 • Z-score anomaly detection

zscores

```
def remove_anomalies(readings, threshold=2.0):
    z_scores = (readings - readings.mean()) / readings.std()
    anomaly_mask = np.abs(z_scores) > threshold
    return readings[~anomaly_mask]
```

Use case: for real data from a file, the standard cleaning pipeline is read -> calculate Z-scores -> filter by threshold -> write -> read back for verification.

Q. Which is better: .npy binary format or .csv text format?

Use **.csv** with **np.savetxt** when people need to inspect the data, open it in Excel, or share it across languages. Use **.npy** with **np.save** when exact precision matters and NumPy will load it again. Text formats may lose precision after several decimal places; binary formats do not.

09 Combining Arrays and Matrix Multiplication

• Core concept

Join patient groups + calculate a weighted score

combine

```
np.concatenate([morning, afternoon]) # Join 1D arrays end to end
np.vstack([matrix, new_patient_row]) # Add a new patient row
np.hstack([matrix, new_day_column]) # Add a new day column

metrics_matrix @ metric_weights
# Multiply each patient's metrics by matching weights and sum.
# This is the same core operation used by every neural-network layer.
```

Use case: combine newly collected data or compress several metrics into one composite score. You will see the **@** operator constantly in PyTorch: each neural-network layer performs a matrix multiplication once per sample or batch.

Common Error Summary

Incorrect	Why it fails	Correct
<code>rates > 100 & oxygen < 95</code>	<code>&</code> has tighter precedence than comparisons and evaluates <code>100 & oxygen</code> first.	<code>(rates > 100) & (oxygen < 95)</code>
<code>row = matrix[0]; row[0]=999</code>	The slice is a view and shares memory with the original array.	<code>row = matrix[0].copy()</code>
Use <code>axis=0</code> for "average per person"	<code>axis=0</code> reduces rows across patients, not within each patient.	<code>axis=1</code>
Print <code>argmax</code> as the maximum value	It returns the position or index, not the value itself.	<code>ids[np.argmax(averages)]</code>
<code>np.mean(oxygen_with_nan)</code>	One <code>nan</code> contaminates the entire result.	<code>np.nanmean(...)</code>
<code>np.random.seed(42) + global functions</code>	Global state is easy for unrelated code to change.	<code>np.random.default_rng(42)</code>

FAQ

Q. How is a mask different from a regular if statement?

`if` usually makes one decision once. `mask = array > x` evaluates every element at once and returns a same-length True/False array that can directly index data. This is the core of vectorization.

Q. Should axis be memorized as 0 or 1?

Imagine a patient-by-time matrix. `axis=0` travels down rows and leaves columns; `axis=1` travels across columns and leaves rows. The result keeps the dimension that was not reduced.

Q. Why does a view affect the original array? Is this a bug?

No. Slicing does not copy by default; it gives another view of the same memory. This avoids expensive copying. Use `.copy()` when independent data is required.

Q. Are np.nan and Python None the same?

No. `np.nan` is a special floating-point value for numeric arrays; `None` is Python's general "no value" object. Use `np.isnan(x)`, not `x is np.nan` or `x == np.nan`.

Q. Which is better: .npy or .csv?

Use `.csv` for people, Excel, or cross-language sharing. Use `.npy` for exact precision and loading back with NumPy.

Q. Why should random numbers use a seed?

A seed fixes the generator's starting point. The same seed yields the same pseudo-random sequence, making debugging and experiments reproducible.

12 Exercises

• Practice problems

1. **E Find high-risk patients:** `ids=[101,102,103,104]`, `rates=[72,108,95,115]`, `oxygen=[98,93,96,90]`. Return IDs whose heart rate is above 100 and oxygen is below 95. Expected: `[102, 104]`. Use Boolean conditions with `&`.
2. **M Find the patient with the highest three-day average:** use `ids=[201,202,203]` and a three-row matrix. Expected ID: `202`. Use `axis=1` and `argmax`.
3. **M Impute missing oxygen values and round:** `[98.0,nan,91.0,nan,96.0]` -> `[98,95,91,95,96]`. Use `nanmean`, `isnan`, assignment, and `rint().astype(int)`.
4. **M Detect and remove anomalies:** remove values from `[72,74,73,130,74,72,20,75]` using a Z-score threshold of 2.0. Expected approximate result: `[72,74,73,74,72,75]`. Use mean/std, the Z-score formula, and negated indexing.
5. **E Safely modify one row:** start with `matrix=[[1,2,3],[4,5,6]]`. Change the first value of the first row to 999 without changing the original matrix. Compare `.copy()` with a view.

13 Review Order and Checklist

• Must master

Boolean mask: `mask=array>x`, `array[mask]`

`axis=0` (across rows/by column) vs `axis=1` (across columns/by row)

a view can change the source; `.copy()` is safe

`argmax/argmin/argsort` return positions, not values

`np.isnan` + `np.nanmean` for missing values

`default_rng(seed)` for reproducibility

• Recommended practice

parenthesize each condition in combined masks

shape alignment with reshape and broadcasting

`diff` for changes; `clip` for range constraints

write the Z-score anomaly formula by hand

distinguish concatenate/vstack/hstack

use `@` for weighted sums

• Next steps

broadcasting with 3D and higher arrays

`np.select` and multi-branch where

structured arrays and mixed dtypes

Pandas DataFrame

scipy statistics and signal processing

vectorized batch operations in ML

• Chapter checklist

- A Boolean mask has the same shape as the source and can index another matching array.
- `axis=0` keeps columns; `axis=1` keeps rows.
- `argmax`, `argmin`, and `argsort` return positions; use them to retrieve an ID.
- Use `nanmean` and `isnan` for missing numeric values.
- Use `.csv` for readability and `.npy` for precision; verify with `array_equal`.
- Every condition in a combined mask should be wrapped in parentheses.
- Ordinary slicing usually returns a view; `.copy()` returns an independent copy.
- `np.diff` finds changes; `np.clip` constrains a valid range.
- Use `default_rng(seed)`, not global `np.random.seed`.
- `concatenate` joins 1D arrays, `vstack/hstack` add rows/columns, and `@` calculates weighted sums.

After completing every exercise, you have turned the concepts of linear algebra - vectors and matrices - and statistics - means, standard deviations, and anomaly detection - into executable code. The next step is to connect this vectorized thinking to **Pandas** for labeled tabular data and **scikit-learn** for direct model input.